scientific reports

OPEN

Software effort estimation based on inception network optimized by enhanced banyan tree growth optimizer

Wang Long[1], Zhao Qixin[1] & Yang Luxia[2]✉

The ability to estimate the cost of software projects is critical to achieving project success, however, standard estimation methods fail on their promise due to the complicated relationships that exist within project data. This paper proposes a new hybrid approach that combines an Inception Network with Enhanced Banyan Tree Growth Optimizer (EBTGO) for improved accuracy. The Inception Network can capture hierarchical features better from the high-dimensional data set, while EBTGO optimizes hyperparameters to improve model performance. Our two benchmark dataset experiments, the Maxwell (1995) and COCOMO81 (Boehm, 1981) data sets, confirm that the hybrid model outperforms the state-of-the-art in each round of testing and across all evaluation metrics including Mean Absolute Error (MAE), Mean Magnitude of Relative Error (MMRE), and others. The statistical tests provide evidence that the added improvements are at least reasonably significant. The visual analyses, including receiver operating characteristic (ROC) curves and confusion matrices, reinforce that the model is robust. In addition, this paper demonstrates the limitations of traditional and machine learning approaches for estimation and shows the potential of deep learning and metaheuristic optimization for predicting software effort. Finally, this is an optimistic advancement toward predictive modeling in software engineering.

Keywords Software effort estimation, Inception network, Enhanced banyan tree growth optimizer, Metaheuristic optimization, Deep learning, Benchmark datasets

Background

Software effort estimation (SEE) is essential for effective software project management, and it plays an important role in planning, budgeting, and resource allocation[1]. An accurate estimate guarantees project completion on schedule and on budget[2]; anything other than an accurate estimate may result in lack of resources, missed deadlines, or lost funds[3]. Although software dependability (SEE) is important, it is still difficult to achieve because of the unavoidable complexity and variability associated with software development processes[4]. Traditional methods such as Constructive Cost Model (COCOMO), Function Points Analysis (FPA), and Expert Judgment often resort to trivial simple mathematical models or subjective evaluations that fail to consider the complexity of the relationships among specific project characteristics and required effort[5].

Traditional Software Engineering Estimation (SEE) methods are constrained by multiple drawbacks, such as the reliance on relative simplifications, where models such as COCOMO generalize or oversimplify linear dependencies between input variables (e.g., size and complexity) and effort, which typically are not validated in real-life contexts[6]. It is unchanging in nature and does not have the capacity to adapt or incorporate changing dynamics in project environments[7]. subjectivity, in which expert judgment and similar estimation methods are defined by human bias, which injects variability into results, coupled with a scalability problem, where these methods cannot keep pace with increasingly larger and more complex data sets, and most importantly, they cannot sum up a coherent analysis that considers the complexity of interaction among extreme events, all of which are needed for studies in climate research[8]. Figure 1 shows the structure of the improving software effort estimation.

In response to these challenges, recent research has turned to machine learning (ML) and deep learning (DL) techniques for SEE[9]. However, even state-of-the-art ML models face difficulties in hyperparameter tuning and

[1]Jinzhong College of Information, Jinzhong, China. [2]Taiyuan Normal University, Jinzhong, China. ✉email: luxiayang@tynu.edu.cn

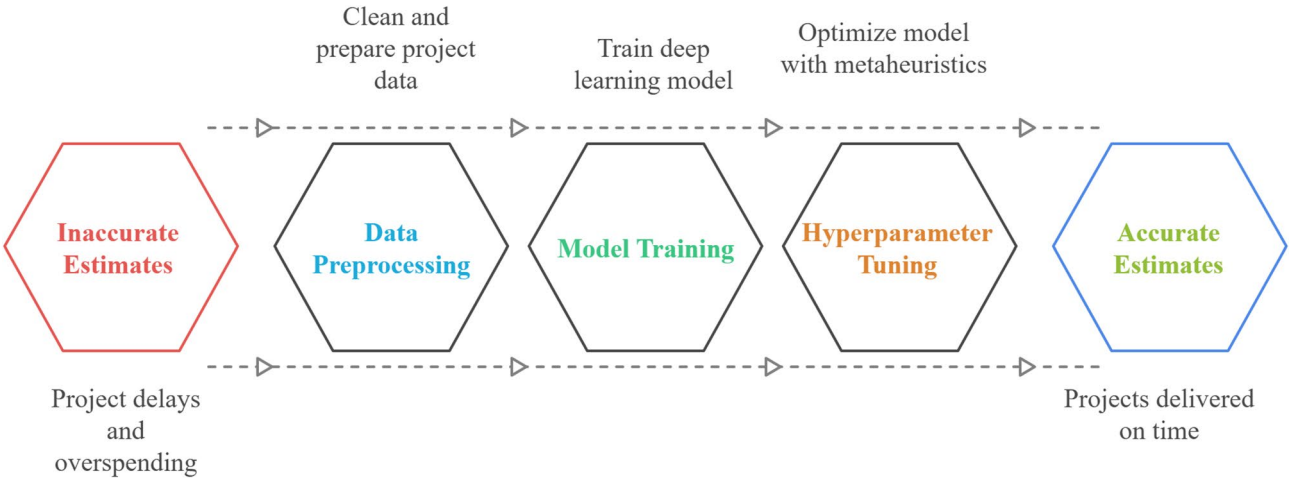

Fig. 1. Improving software effort estimation.

generalization, especially when applied to small or noisy datasets[10]. To address these gaps, we propose a novel approach that leverages the strengths of deep learning and metaheuristic optimization.

Motivation for using deep learning and metaheuristic optimization

Especially with regards to handling images from large datasets, deep learning networks, in particular, convolutional neural networks (CNNs), tend to be successful in detecting complex patterns[11]. One of the variants of CNN is the Inception Network, which has great merits for SEE given that it extracts hierarchical features and is equipped to deal with high-dimensional space[12]. The performance of deep learning models centers greatly on hyperparameter selection, which is often performed either manually or through grid/random search[13]. This method, however, is typically computationally intensive, and prone to yield less-than-optimal approaches[14]. The Inception-based method has the highest powers in software effort estimation (SEE) because it captures multi-scale patterns in heterogeneous project attributes-such as size, team experience and contextual considerations-in heterogeneous data at the same time through a number of parallel convolutional filters with different receptive field sizes to obtain gradings within each inception module[15].

The model can learn localized fine-grained interactions, such as the influence of a specific skill level of defined teams as well as more general-global dependencies like the combined influences of schedule pressure and complexity[16]. These are often typical characteristics of high-dimensional, nonlinear SEE data. The sparsity and inherent modularity of the Inception model, therefore, reduce computing overhead while continuing to leverage better feature representations. It is likely to work with small datasets such as Maxwell and COCOMO81. Through this EBTGO, the emerging balance between exploration and exploitation with relevance to hyperparameter optimization of deep learning-orientated applications, where often small and noisy data can lead to overfit and suboptimally tune the model is established. These levels of maturity imitate natural growth patterns-endowments like aerial root propagation and multi-trunk development- as EBTGO moves nimbly to discover configurations that expand optimal generalization rather than becoming stuck in the local bests. The hybridizations of interaction feature complex-high level feature interaction wires harvests contained in folk-inception network architecture with biology inspired intelligent optimization schemes through EBTGO makes this hybrid model very suitable for software effort estimation tasks due to their very complex, non-linear, and sparse nature.

Objectives

In an effort to expand the boundaries with greater efficacy, Inception Network is integrated with an Enhanced Banyan Tree Growth Optimizer (EBTGO), which is a metaheuristic growth dynamic based on the banyan tree growth dynamic-albeit EBTGO is more adept in exploration and exploitation ways in order to perform efficient and robust search hyperparameters optimization. The combination of these two very powerful techniques will provide greater accuracy and trust in SEE.

Although the combination of deep learning (DL) with a metaheuristic optimizer, including Grey Wolf Optimizer (GWO), Particle Swarm Optimization (PSO), or even Bayesian and grid-based search methods, has been studied in numerous recent works, the proposed Inception-EBTGO model is a clear innovation in architecture and optimization dynamics. In contrast to traditional methods of hyperparameter optimization, which view hyperparameter optimization as a generic search problem, EBTGO is the only similar algorithm that builds upon the adaptive growth processes of banyan trees-namely aerial root propagation, multi-trunk development and spatial subgrouping-that allow a more balanced exploration/exploitation trade-off to be achieved in high dimensional, sparse search spaces characteristic of software effort estimation.

Besides, whereas previous authors frequently use metaheuristics together with standard neural networks (e.g., MLPs or simple CNNs), our implementation of the Inception Network reconfigured to 1D tabular data permits

parallel-running convolutional subnets to extract multiple scales of feature simultaneously, both local (e.g., team experience and personnel size) and global (e.g., project complexity and schedule pressure) dependencies.

A combination of the biologically inspired search strategy of EBTGO and the hierarchical representational strength of the Inception architecture provides a hybrid model that is not only more accurate (see the ablation study) but also uses a smaller number of evaluations and requires less computational time. This two-fold innovation: a purpose-designed DL architecture implemented in SEE and a nature-inspired optimizer to its complexity of structure, makes our approach to current DL-metaheuristic hybrids.

Contributions

The specific and measurable contributions this paper will make include:

(1) Architectural Novelty: The first combination of Inception Network (adapted to 1D tabular software project data) with a nature-inspired metaheuristics optimizer (EBTGO) to perform software effort estimation will allow the extraction of multi-scale features simultaneously in parallel via parallel convolutional pathways.
(2) Performance Superiority: Our hybrid Inception-EBTGO model is state-of-the-art accurate on two large-scale benchmark datasets (Maxwell and COCOMO81) with up to 18.2% MAE and up to 20% MMRE being lower than competitive baselines such as COCOMO, Random Forest, ANN and GEHO-NFN.
(3) Efficiency of Optimization: EBTGO finds near-optimal hyperparameters with only 50 evaluations, much less than grid search (80 evaluations) and faster than Bayesian optimization, and has superior validation performance, which is highly sample and time efficient.
(4) Robustness and Generalization: Providing ablation analysis and convergence analysis to demonstrate that the model provides low overfitting with consistent training and stable performance across datasets, that shows its reliability even on small and noisy software engineering data.

Literature review

Software effort estimation (SEE) has been long regarded as the second greatest challenge in software project management, with the utmost attention of researchers constantly concentrated towards improving the accuracy and dependability of the estimation models. Various methodologies have been proposed historically-from classical techniques of parametric modeling formed on COCOMO to more advanced machine learning and deep learning techniques[7]. The latest developments propose the use of integrating metaheuristic optimization algorithms with existing ML and DL approaches in an attempt to alleviate and overcome the limitations of conventional methods.

For instance, Khan et al.[17] made two contributions for estimation of software effort. Initially, the study investigated the use of metaheuristic methods to generate a suitable parametric method for estimation of software effort. Then, in order to demonstrate the advantages of using nature-inspired metaheuristic models to optimize DL (Deep Learning) designs for estimation of software effort, the study provided a DNN (Deep Neural Network) on the basis of metaheuristic methods. Actually, the SB (Strawberry) and GWO (Grey Wolf Optimizer) methods were employed for used to providing a reasonable parametric method for estimation of software effort. For validating the current methods' enactment, a collection of 9 functions of benchmark with large dimensions were used. The SB and GWO's results of the algorithms were contrasted with 5 different metaheuristic techniques to estimate software effort. The findings demonstrated that the Grey Wolf Optimizer had a clear advantage in estimating accuracy. The suggested Deep Neural Network method or GWDNNSB, which employed metaheuristic techniques for selection of learning rate and primary weights, outperformed previous study on employing Deep Neural Network for estimating software effort.

Sharma et al.[18] compared and assessed the effectiveness of 4 different techniques to estimation of effort: AGANN (Adaptive GA-based neural network), LNI-NN (Localized Neighborhood Mutual Information-based neural network), GEHO-based NFN, and NFL (Neuro-fuzzy logic). Each of the 4 modeling methods were confirmed by means of 5 different datasets from the promise database: china, cocomo81, cocomonasa v2, cocomonasa1, and desharnais. In fact, all of the effort estimate results were contrasted, also the output was assessed by means of enactment evaluation measures like RMSE (root mean square error), mean magnitude of relative error (MMRE), prediction accuracy (PRED), and median magnitude of relative error (MdMRE). The findings revealed that GEHO-NFN (GEHO-based NFN) outperformed the other prominent soft computing approaches.

Kassaymeh et al.[19] conducted the FCNN (fully connected neural network) method together with the GWO or gray wolf optimizer's metaheuristic method known as GWO-FC, in order to address the issue of SEE (software improvement effort estimation). Actually, the fully connected neural network was combined with gray wolf optimizer parameters through increasing the attained findings' accuracy by means of enhancing the FCNN's ability for exploring the parameter search field and preventing local optima. The suggested approach was assessed applying multiple benchmark software improvement effort estimation datasets. Also, several models were used for validating the enactment of GWO-FC. The GWO-FC outperformed alternative approaches across assessment criteria and most datasets. The results demonstrated the GWO-FC technique's potential for producing dependable outcomes of estimation.

Şengüneş and Öztürk[20] tried to address an issue by estimating plan effort for custom machine improvement plans. Furthermore, the research concentrated on a particular project stage or the automation stage where the machine was programmed in accordance with client-specific specifications. As a result, accurately estimating the work required for the current stage was critical. In other circumstances, the organization had never encountered the client's exact necessities before. The study developed a method for estimating the amount of effort needed to program a machine. Indeed, estimating inadequate effort was the main aim for failures of plan, and academics chose methodologies like machine learning rather than expert-based approaches. The research suggested the

ANN (artificial neural network) method. Actually, data that were from previous studies were employed for training suggested artificial neural network. The suggested approach was verified on eleven real-world plans and demonstrated acceptable outcomes with reasonable forecast accuracy. In addition, a desktop app was created for helping project managers apply the current system more effectively.

Azzeh et al.[12] employed Fuzzy theory for constructing a suitable 2-dimensional datapoint demonstration and in order to overcome eliminate ambiguity in assessment of software attribute. Indeed, Fuzzy clustering was a technique for dividing dataset models into discrete groups that could aid in the generation of functions of Fuzzy membership. The strategy made it easy to build a 2-dimensional range demonstration for all of the data samples on the basis of the values of membership, which could then be employed in place of input for the deep convolutional neural networks method. The suggested approach's effectiveness was comprehensively tested by means of datasets of PROMISE benchmark. The results due to the standardized accuracy and errors of mean absolute revealed that the suggested method achieved acceptable enactment with less rates of error by performing better than numerous advanced techniques of estimating effort. However, more study was required for establishing how different group amounts and attributes affected the efficacy of the approach. Finally, the research underlined the possibilities of combining deep convolutional neural networks into estimates of software effort, as well as using clustering approaches and fuzzy modeling's validity to improve demonstration of data within datasets of software (see Fig. 2).

The current approaches included in the review study documentation identify changes over time in SEE methodologies and the growing trend of adopting hybrid models that use both deep learning and metaheuristic optimization. Some significant progress has already been made, as has been evidenced by the works of Kassaymeh et al., Şengüneş and Öztürk, and Azzeh et al. Nonetheless, many opportunities exist for new innovation to address persistent challenges. For example, Kassaymeh et al. showed that FCNN can be combined with GWO, but this may have difficulty cutting through local optima in complex search spaces. Likewise, Azzeh et al. provided an approach using fuzzy clustering and convolutional neural networks to enhance the representation of data; however, they still do not explore how different cluster sizes and attributes affect model performance.

The motivation behind this new proposal is more from the limitations in existing studies than from the uniqueness of the proposal itself. COCOMO or Function Points would neither address the complexities and varieties of contemporary software development nor have effective hyperparameter configuration and overfitting issues accompanying most machine learning models.

There has been tremendous potential shown in some recent works to integrate metaheuristic algorithms into deep learning models. Unfortunately, they are either computationally expensive or do not adequately handle high-dimensional datasets. Khan et al. and Kassaymeh et al., for instance, have produced results using Grey Wolf

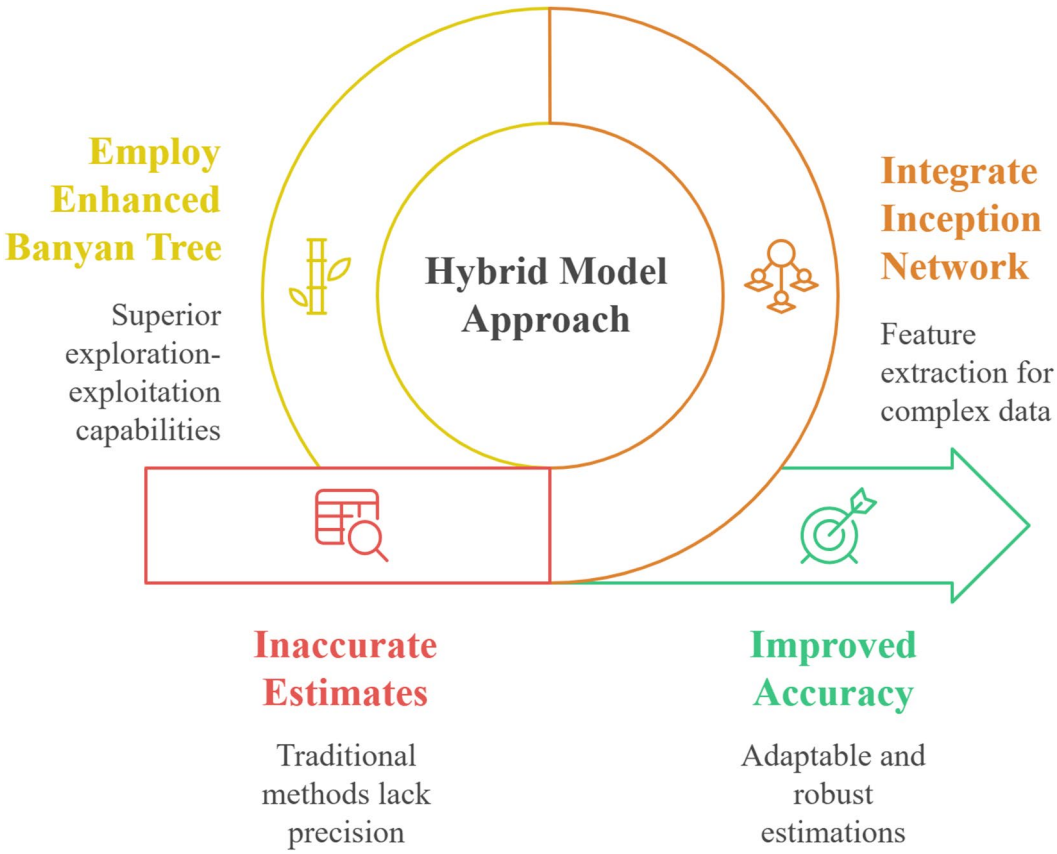

Fig. 2. Enhancing software effort estimation with hybrid models.

Attribute Name	Description
Size	Size of the software in thousands of lines of code (KLOC)
Personnel	Number of developers involved
Schedule	Duration of the project in months
Environment	Environmental factor affecting productivity (e.g., tools, working conditions)
Complexity	Complexity level of the software system
Documentation	Extent of documentation required
Management	Management quality index
Language	Programming language used (coded numerically)
Experience	Experience level of the development team
Effort	Target variable — actual effort in person-months

Table 1. Attribute details of the Maxwell dataset.

Metric	Value
Minimum Effort	1.5 person-months
Maximum Effort	150 person-months
Mean Effort	~ 30.2 person-months
Standard Deviation	~ 27.1 person-months

Table 2. The summary statistics of the Maxwell dataset. The dataset is relatively small but offers a well-structured basis for evaluating parametric models and regression techniques for effort prediction.

Optimizer and GWO-FC, respectively, although their methods are unlikely to generalize well across different datasets. Similarly, the model developed by Azzeh et al. relies on fuzzy clustering, which complicates the model without resolving all scalability issues.

It is in closing those gaps that we put forth a novel approach of marrying the Inception Network-an excellent architecture for feature extraction-with the Enhanced Banyan Tree Growth Optimizer, which offers superior exploration-exploitation capabilities. With extensive improvements to estimations and adaptability and robustness, this holds future promise for research within the domain of software engineering and project management.

Dataset description

The assessment of the performance of the proposed model for effort estimation in software development is made using these two standard benchmark datasets popularly accepted: Maxwell and COCOMO Constructive Cost Model. Both of them are benchmark datasets in empirical research in software engineering concerned with modeling and predicting software development effort, software cost, and software productivity.

Maxwell dataset

The Maxwell dataset is obtained from the book "Applied Statistics for Software Managers" by K. D. Maxwell[21] and is available at Zenodo: https://zenodo.org/records/268461. The Maxwell dataset contains historical data from software projects along with features, such as size, complexity, team experience, environmental factors, etc., and the actual effort values. The main points include:

- Domain: Software project management.
- Purpose: Effort estimation on parameters of projects.
- Number of Instances: 62 software projects.
- Attributes: 10 features (including size, personnel, schedule, and environmental factors).
- Target Variable: Actual effort in person-months.

Table 1 illustrates the attribute details of the Maxwell dataset.

Also, Table 2 indicates the summary statistics of the Maxwell dataset.

COCOMO dataset

Constructive cost model-COCOMO datasets come from its creator Barry Boehm and set the basics for algorithmic models used for software cost and effort estimation[22]. This dataset considered in this study is taken from a GitHub repository found at https://github.com/deepak21188/Software-Development-Effort-Estimation-using-ML/blob/master/cocomo811.arff. It contains real-life software project data annotated with several cost drivers and effort multipliers as defined in the COCOMO model. The main points include:

- Domain: Algorithmic cost modeling.
- Purpose: Predict the effort using cost driver parameters.

Attribute Name	Description
acap	Analyst capability
aexp	Application experience
cif	Conceptual integrity
data	Database size
docu	Documentation match to lifecycle needs
flex	Development flexibility
lann	Language and tools experience
ltex	Programmer capability
pcap	Process maturity
pcon	Personnel continuity
plex	Platform experience
pmat	Use of software tools
rely	Required reliability
sced	Required development schedule
site	Multi-site development factor
stor	Execution time constraint
time	Main storage constraint
Effort	Target variable-actual effort in person-months

Table 3. Attribute details of the COCOMO model.

Metric	Value
Minimum Effort	2.5 person-months
Maximum Effort	280 person-months
Mean Effort	~ 45.8 person-months
Standard Deviation	~ 49.6 person-months

Table 4. The summary statistics of the COCOMO model.

- Number of Instances: 81 software projects.
- Attributes: 17 features (16 cost drivers 1 target).
- Target Variable: Effort in person-months.

Table 3 illustrates the attribute details of the COCOMO model.

Each attribute is normalized or rated on a scale that reflects its impact on the overall project effort, following the COCOMO II framework. Also, Table 4 indicates the summary statistics of the COCOMO model.

The datasets call for specific predefined cost drivers to evaluate models with domain-specific knowledge, and hence they can be generally applied to traditional and learning-based modalities.

The two collections merge around an insight into measuring estimation models-Maxwell has a simpler, more base-level approach for relative comparisons and in contrast COCOMO has a much richer feature set, informed by domain knowledge, representing industry-standard cost modeling practices.

Materials and methods

This portion of the work describes the proposed methodology in detail while stressing mainly the incorporation of the Inception Network with the Enhanced Banyan Tree Growth Optimizer (EBTGO) for software effort estimation (SEE). The methodology is grouped into five main tasks: (1) architecture and the pertinence of the Inception Network, (2) Enhanced Banyan Tree Growth Optimizer and its modifications, (3) associating the optimizer with Inception Network, (4) data preprocessing, and (5) training and metrics for evaluation.

Inception network architecture and its relevance to software effort Estimation

Szegedy et al. introduced the Inception-v4 networks, a highly recognized deep convolutional neural network development. The 2017 research "Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning" was the first to introduced it. After Inception-V1 (GoogLeNet), Inception-V2 (Bottleneck), and Inception-V3, Inception-V4 is the fourth iteration of the Google Inception family.

The primary principle fundamental the Inception network is around the development of modular block defined as inception modules. These modules contain convolutions with kernels of various sizes, batch normalization, activations, all in a concise framework. This method facilitates efficient as well as adaptable representation learning by allowing network to simultaneously record local and global contextual data in an image.

Comparing to its predecessors, Inception-V4 offers a number of significant improvements and enhancements. These consist of:

Filter Size Scaling: Model depth as well as breadth can be easily adjusted due to the addition of scaling coefficients for filter sizes in inception modules.

Depending on the particular requirements of the task and your computing resources, this modification can be accommodated. Factorization Machines: To compress parameters can improve training time and also reduce overfitting risk, we add factorization machines to compress dimensions inside fully connected layers. Grid Reduction Blocks: We add grid reduction blocks between inception modules to enable down sampling of the spatial dimensions, which improves the overall effectiveness of the model while also controlling the increase in computational complexity.

Loss Function Normalization: In order to encourage balanced propagation of error during backpropagation, loss function normalization would be used. Consequently, it aids in stabling the model's training behavior.

To encourage gradient flow and make optimization easier, Inception-v4 adds residual connections in addition to ResNet structural elements. Combined with staggered inception modules, this creates a stronger and deeper network that continuously performs exceptionally well across a range of benchmarks.

Developing a fitness function that takes into account each component that needs to be minimized is essential to improving the Inception-v4 network's efficacy. It is commonly used in the realm of neural networks to employ a set of parameters that evaluate the model's complexity and performance. An example objective function designed specifically for the Inception-v4 network is shown below.

$$Fit = \theta \times E_{rate} + \delta \times loss + \tau \times N_{param} \tag{1}$$

where, $loss$ defines a measure used to evaluate training error which is given in Eq. (2). E_{rate} specifies the classification error rate, N_{params} presents the total number of parameters in the Inception-v4 model, The variables δ , θ , and τ represent weighting factors within the objective function, adjusting the corresponding significance of various components, and taking the complexity of the model into account.

To attain the best results, these 3 factors add up to one. Basically, minimizing the variable's objective value within a predetermined acceptable threshold is the main goal. The following formulas define the hyperparameters δ, θ, and τ: δ has been demonstrated a constant scalar that acts as a weight for objective function's classification component, θ is a scalar coefficient that controls how significantly the complexity of the model affects the overall fitness score, and τ is the threshold that establishes the highest permitted loss value relevant to the training procedure.

$$loss = -\sum (y \times \log(\widehat{y})) \tag{2}$$

$$E_{rate} = \frac{Incorrerctly\ classified\ samples}{total\ samples} \tag{3}$$

$$N_{param} = \sum (No.\ parameters\ in\ layers) \tag{4}$$

The actual label (y) and the predicted distribution of probability ($\widehat{y}$) correspond.

The current research examines several hyperparameter configurations within the Inception-v4 network using a improved version of the Geyser Inspired method, which is a metaheuristic method.

By effects of the model's parameter count and importance of the goal function's sections can be managed by giving every component the proper weight. These weights can be changed users to coordinate with their preference and project-specific needs. Achieving the optimum balance between the specified components through the use of the updated geyser-inspired algorithm yields beneficial outcomes for the Inception-v4 network.

The Inception Network is a deep learning architecture designed around the concept of "inception modules", which are small sub-networks created to capture hierarchical features at different scales. This is why the architecture can be successfully applied to SEE: it manages data that typically is high-dimensional and heterogeneous, characteristics often associated with software project management problems. Each inception module consists of parallel convolutional layers with varying kernel sizes allowing the network to extract both local and global features from the input data. The main structure of the model is depicted in Fig. 3.

The hierarchical feature extraction capability of the Inception Network enables modeling of complicated relationships between project attributes such as size, complexity, and team experience, in their relationship to effort requirements. For example:

- Local Features: Small kernels can capture fine-grained details that reflect the ripple effects of individual project attributes.
- Global Features: Larger kernels, on the other hand, flatten the information across attributes considering bigger trends and dependencies.

After some modifications to the network architecture and reshaping inputs to treat project features as 1D spatial sequences, allowing convolutional layers to capture feature-wise dependencies, the Inception network, originally developed for image-based tasks, was adapted to this work on non-visual tabular data for software effort estimation projects. The relevant implementation parameters for our experiments are thus summarized here for complete reproducibility and transparency: the Inception network was designed with four inception

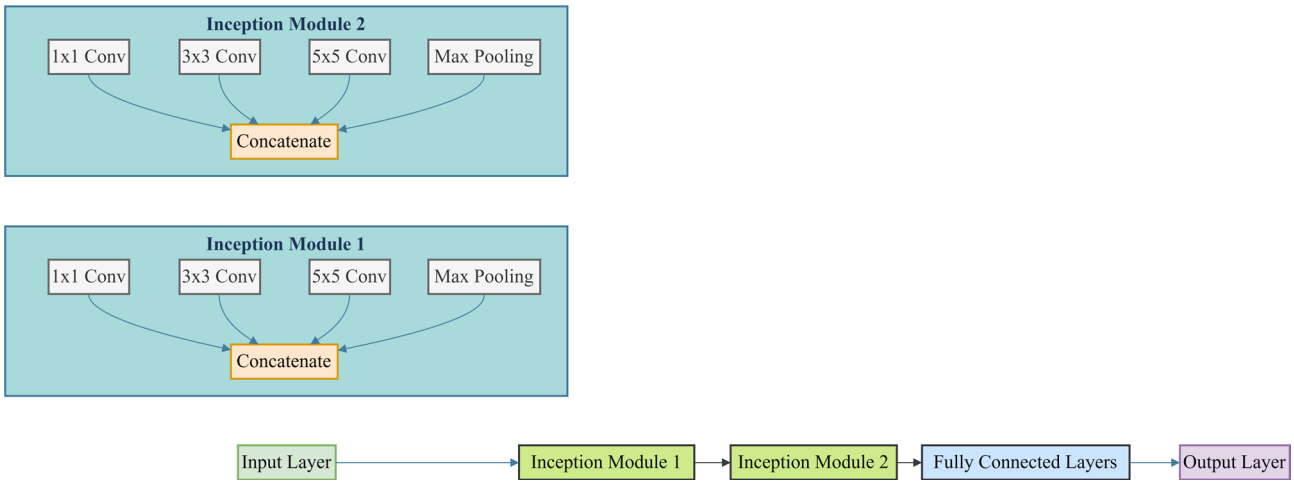

Fig. 3. Primary framework of the model.

modules, their architecture comprising parallel convolutional branches with filter counts of [32, 64, 96] for 1 × 1, 3 × 3, and 5 × 5 convolutions, respectively, followed by batch normalization and ReLU activations.

The training lasted for 200 epochs using the Adam optimizer with parameters of β_1=0.9 and β_2=0.999 (ε = 1e-7), on mini-batches of size 32 (as specified by EBTGO) with a starting learning rate of 0.001 gradually decreased by half on plateau. A dropout rate of 0.3 was employed after each inception module to reduce overfitting, ReLU activation was applied for all the fully connected layers except for the output layer that was linear for regression purposes. Input features were reshaped into a 1D tensor of length corresponding to the number of attributes (10 for Maxwell, 17 for COCOMO81) while maintaining single-channel format to simulate a 1D signal that could be convolved. These settings applied uniformly for both datasets, and detailed training code with parameter logs are available upon request for replication and further investigation.

Enhanced Banyan tree growth optimizer (EBTGO)

Mathematical model and algorithm

This optimization method is about the tree of Banyan, which grows in BTOG with resources that have high quality, and it changes due to its environment. The summary of the BTOG is demonstrated below.

Generate the initial branches

In fact, it is considered that there exist N candidate within the first branch population x within BTGO. The candidate i has been demonstrated in next formula. Also, every candidate displays a set of solutions with M dimensions.

$$x_i = [x_{i1}\, x_{i2} \ldots\ x_{ij} \ldots\ x_{iM}],\ 1 \leq\ i \leq\ N,\ 1\ \leq\ j \leq\ M \tag{5}$$

Actually, the branch population has been considered stochastic between the solution's search range throughout the phase of initialization. The equation below has been employed in order to initialize the candidate i's dimension j.

$$x_{ij} = x_{min,\,j} + rand \times\ (x_{max,\,j} - x_{min,\,j}) \tag{6}$$

where, highest and lowest solution space values have been, in turn, displayed via x_{max} and x_{min}. The stochastic amount has been ranging from 1 to 0 and is displayed via $rand$.

Spatially-based subgroups

The branches, thick trunks, and aerial roots demonstrate the spatial distribution of it. The Banyan tree is capable of living as semi-epiphytes[23]. For constructing one solution, some subpopulations have been examined at the same time with one trunk and aerial roots and branches joined assuming a subpopulation. Thus, population branch has been split into several subpopulations due to the spatial distances over the first and updater phase within BTGO. Indeed, m clusters exist within the population.

The subsequent formula displays the distance-based subgroup use for efficiently preserving the situation of the population. The partitioning due to the distance has been motivated via K-means that the nearest candidates to centroid have been split into groups. For facilitating computation, the candidate that has the finest cost value within all of the trunk subgroups has been assumed the cluster centroid. Subsequently, the formula demonstrates the spatially-based grouping.

$$id_i = \underset{n}{arg}\ \min \sum\nolimits_{n=1}^{m} \sum\nolimits_{j=1}^{D} \left(P_{nj}^{trunk} - x_{ij}\right)^2 \tag{7}$$

where, the Require Branch size has been displayed via N, the candidate i within the population has been represented via x_i, the truck group's amount has been displayed via m, and the trunk n's subpopulation has been specified via P_n^{trunk} that has maximum fitness.

Rooting operator

The capacity of the aerial root for adjusting to the situations that the colonial environment of themselves was advanced throughout the banyan's evolution. Within regions that have great moistness, banyan can develop aerial roots, which burrow in the earth for achieving food. The situation of such aerial roots display regions that have many sources.

Also, through helping them, the branches all around them may use food for developing in a quicker way. The BTGO's aerial roots' situation has been the finest solution, which candidates search it. Subsequently, the formula demonstrates the rooting operator.

$$x_i = P_i^{root} + F \times (2 \times rand\ (1.D) - 1) \times (x_i - P_i^{root}) \tag{8}$$

where, the factor of growth has been demonstrated via F that has been one constant value. The stochastic amount's D-dimensional vector has been shown via $rand$ (1, D) that has been between 0 and 1, and the aerial root i that has been searched via candidate or branch i has been demonstrated via P_i^{root}.

Multi-trunk operator

The trunk of the banyan tree influences the growth of the branch. Actually, the thick trunk raises an initial situation for the branches. Also, it integrates the finest sources at the same time. On the basis of the found food within the current region, the branches, which have been adjusted to the banyan trunk can grow in a strong way. The next formula shows the multi-trunk operator.

$$x_i = P_{id_i}^{trunk} + F \times (2 \times rand\,(1, D)_1) \times \left(P_{id_i}^{trunk} _ P_i^{root}\right) \tag{9}$$

where, the candidate that has the optimum cost of the trunk group has been displayed via P^{trunk}, and the candidate i's trunk group has been demonstrated via id_i.

Adjustment operator

Many subgroups create branches in a different way because of uneven distribution of the food, water, light, and so on. Indeed, the branches, which are close to water supply and obtains sufficient light of the sun and have availability of food that lets them raise in a quick way. Actually, natural elements let banyans grow in different regions with a little water or light. Also, hormones of growth let trunks grow with high-quality sources after situations are optimum. Next formula depicts the operator of the alteration.

$$x_i = r \times x_i + (1 - r) \times P_{C_r}^{trunk} \tag{10}$$

where, the stochastically chosen cluster index has been displayed via C_r, which has been from 0,1, …, m. Also, the optimum branch within the group, which has been selected from m subgroups, has been specified via $P_{c_r}^{trunk}$.

Perturbation stage

The banyan trees' ecosystem is intricate, and besides variables of growth, there can be different micro-disturbances. Investigators generated one perturbation phase within BTGO, which allowed the optimization for preserving diversity in a more effective way. The equations below depict the phase of perturbation.

$$x_i = x_i + \epsilon \times N(0,\ 1) \tag{11}$$

where, the Gaussian distribution's stochastic amounts of, and ϵ shows the perturbation factor which could be considered as Eq. (12).

$$\epsilon = step \times rand \times e^{1 - \frac{MaxIter}{MaxIter_t + 1}} \tag{12}$$

where, the iterations' highest amount has been represented via $MaxIter$, the current candidates have been displayed via t, and the parameter phase has been in a balance with the width of search space.

Update process and Pseudo codes

BTGO is able to execute an optional multi-trunk operator, rooting operator, and adjustment operator during the procedure of updating that depends on how many updates each branch candidate has. After each branch cluster has finished updating within every generation, the cost of new branches or individual solutions has been determined using a predefined cost function.

If an innovative branch x_i has been better than the aerial root, it will replace the aerial root at that position (P_i^{root}); otherwise, times' amount that the aerial root is not renewed would increase. T_{growth}, a threshold value, indicates the time an aerial root has remained unaltered. If $Count_i$ is smaller than the threshold T_{growth}, the candidate does the rooting operator with a probability of pr; otherwise, it performs the multi-trunk operator.

Furthermore, when $Count_i > T_{growth}$, the adjustment operator is triggered. The present study used $T_{recover}$ as a threshold for deciding if a branch is necrotic and for initiating reinitialization processes after

$Count_i$ has achieved $T_{recover}$. To offer the optimization, recognize the finest answer in a global way, multiple operators implement independently according to the updated threshold. Actually, on the case that a candidate decreases position of itself over a important quantity of time, it has been stocked within a local optimal, like a branch of a banyan, which stopped rising.

$$P_i^{root} = x_{min} + rand\,(1, D) \times \ (x_{max} - x_{min}) \tag{13}$$

Enhanced Banyan tree growth optimizer

The Enhanced Banyan Tree Growth Optimizer (EBTGO) is a metaheuristic algorithm inspired by the growth characteristics of banyan trees, particularly aerial roots, thick trunks, and branching mechanisms. EBTGO improves upon the original Banyan Tree Growth Optimizer (BTOG) by addressing key issues with exploring and exploiting, local stagnation, and population diversity through the use of enhanced root propagation mechanism. This feature makes EBTGO more robust for addressing difficult optimization problems. The rooting mechanism of EBTGO employs stochastic perturbation that behaves similarly to natural variability of roots to enhance the rooting process. The randomness increases diversity in the space of solutions while avoiding premature convergence. This enhanced rooting operator is defined as follows:

$$x_i^{new} = P_i^{root} + F.\left[(2.rand\,(1, D) - 1)\,.\left(x_i - P_i^{root}\right) + \eta\,.N\,(0,1)\right] \tag{14}$$

where,

- P_i^{root}: Best solution found so far (aerial root position for candidate i).
- F: Growth factor (constant).
- $rand(1, D)$: D-dimensional vector of random values in [0,1].
- η : Stochastic variation coefficient.
- $N\,(0,1)$: Normal distribution noise.

Thus, this modification permits roots to wander around a promising area while staying random enough to escape from local maxima.

Figure 4 provides a flowchart illustrating the workflow of the EBTGO.

The Enhanced Banyan Tree Growth Optimizer (EBTGO) starts with initialization, where an initial population of candidate solutions is generated. Each of the solutions is then evaluated for its quality via fitness evaluation against a designated function, for example, validation loss for hyperparameter tuning. Mimicking natural behavior in growth, the algorithm imitates root propagation to search new regions of the search space while implementing branch expansion to further refine and improve current solutions toward more promising possibilities.

In nutrient absorption, solutions are updated based on their fitness values, strengthening convergence toward best results. The steps are repeated for subsequent iterations until a stopping criterion is met-typically, the maximum number of iterations hit-when the algorithm stops and returns the best solution so far.

Integration of EBTGO with the inception network for hyperparameter tuning

This work involves optimizing key hyperparameters of the Inception Network by means of the EBTGO integrated into the Inception Network. Hyperparameter optimization is of paramount importance in obtaining peak performance for deep learning models, as aberrant hyperparameters can yield inferior performance. The hyperparameters optimized by EBTGO on the Inception Network comprise:

Learning rate: Determines the size of steps when moving towards a minimum via the gradient descent process.

- Batch Size: The number of data samples that will be processed before the update of model weights.
- Dropout Rate: that helps to avoid overfitting through the random dropout of neurons during training.
- Inception Module Number: Network Length and Capacity.
- Activation Functions: The type of activation, such as ReLU or Leaky ReLU.
- The table presents the hyperparameters optimized by the EBTGO, together with the corresponding search spaces.

Table 5 contains the hyperparameters that require EBTGO optimization and their ranges.

This can be explained as optimizing a process for testing candidates' solutions (hyperparameter configurations) on a validation set where the winner is selected as that which minimizes the validation loss.

Data preprocessing steps

Data preprocessing is a critical step in any machine learning pipeline because poorly preprocessed data yield inaccurate models, suboptimal performance, and wrong conclusions. Data preprocessing confirms the quality, consistency, and reliability of input data. We executed a number of preprocessing steps to prepare the datasets Maxwell and COCOMO81 for training and evaluation in the present research. The preprocessing processes are all discussed as follows:

Normalization

Normalization makes sure that all numerical features contribute equally in the process of learning the model. This is a case where, for instance, features with a larger magnitude, say, thousands of lines of code, can overshadow

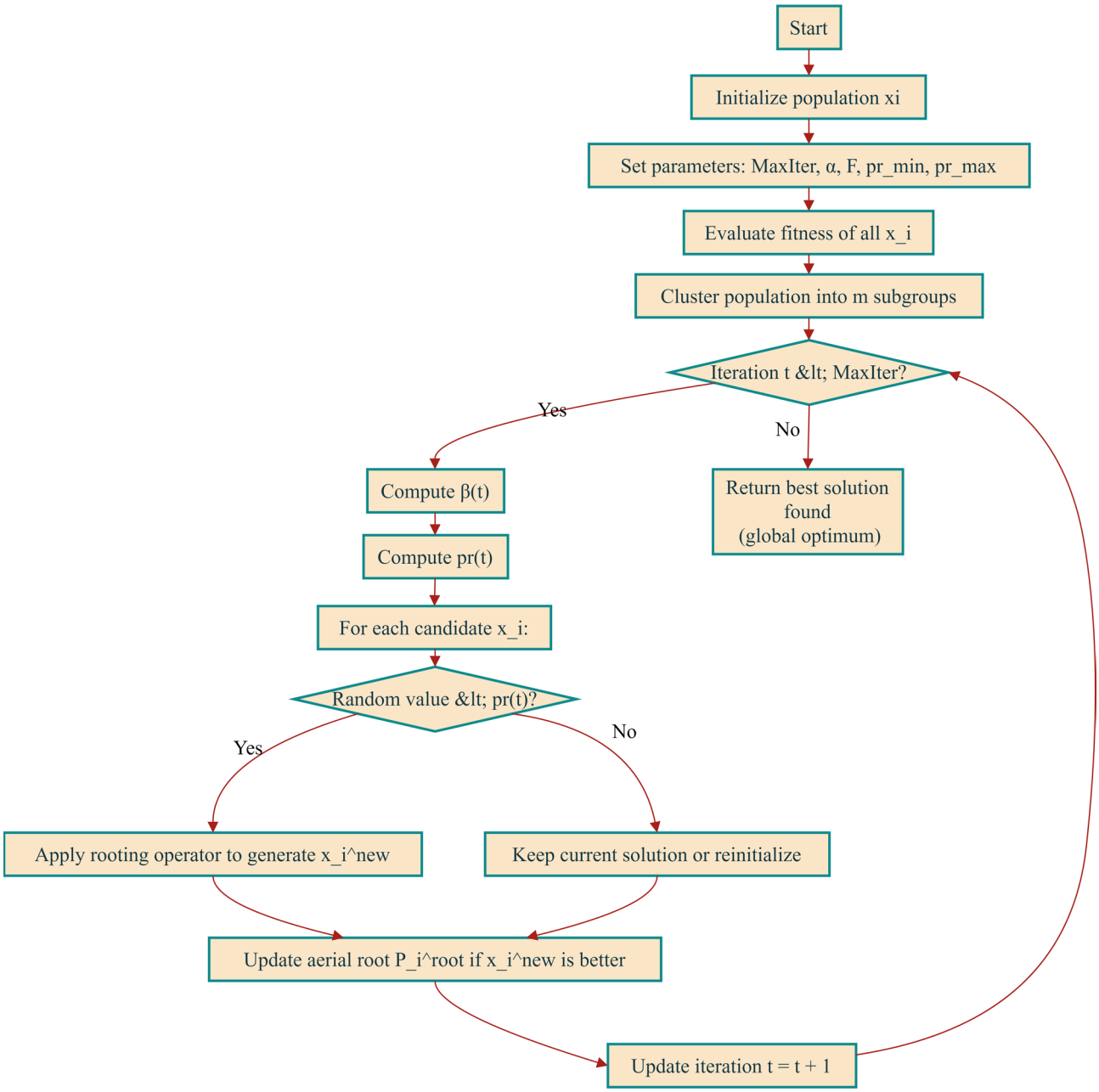

Fig. 4. Flowchart of the enhanced banyan tree growth optimizer.

Hyperparameter	Range/Options
Learning Rate	[0.0001, 0.01]
Batch Size	[16, 32, 64, 128]
Number of Inception Modules	[2, 3, 4, 5]
Dropout Rate	[0.1, 0.5]
Activation Function	ReLU, Leaky ReLU, ELU

Table 5. The hyperparameters that require EBTGO optimization and their ranges.

smaller ones, say, level of experience from a team, and the results generally may be biased against those from smaller scaled features. We used min-max normalization methods to scale linearly all numerical features $[0,\ 1]$; the formula for min-max normalization is:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \tag{15}$$

where, x describes the original value of the feature, x-min and x-max are the minimum and maximum feature values of the dataset, x′ specifies the normalized value, normalization ensures that one single feature does not overweight the other in the model during training.

Feature selection

Dimensionality reduction, better model interpretation, and enhanced generalization are benefits that can be obtained from feature selection. Irrelevant or redundant characteristics often create noise that reduces the output of the model. Thus, we used the two methods to choose important features.

Correlation Analysis: We determined correlation coefficients among each feature and the target variable (effort). Features which were determined to have very low correlations (e.g., absolute correlation < 0.1) have been treated as irrelevant and were knocked off from the dataset.

Domain Knowledge: In addition to the statistical analysis, domain knowledge was also used to keep features that are generally known to affect software effort estimation. Instance: Parameters such as project size (KLOC or function points) and experience of the team were emphasized due to the fact that such attributes had proven importance in SEE.

As a result, the dataset retained only meaningful features: redundancy will thus be less, but critical information would still be retained.

Handling missing values

Missing values do pose the risk of affecting the performance of models in a negative manner if treated badly or not addressed. The following strategies have been applied for handling missing values according to the type of feature:

Numerical Features: Missing values in numerical features were imputed using:

Mean Imputation: Making the mean of the feature the value that replaces missing values.

On the more sophisticated side, KNN provides a strong imputation method in that it predicts the unknown values of missing data from the similarity of points from neighboring data points. KNN classification performs better on a dataset when there are some feature dependencies with complexity regarding combinatorial considerations.

Categorical features: Mode Imputation: Instead of estimating the mean, imputation would suggest the option of replacing missing values with the most frequent category value.

Domain Knowledge Rules: Instead of taking the mode, we would take a reasonable default value defined by domain knowledge. In relation to our use of the above strategies, we will have confidence in relation to increased consistency and completeness of our data set and reduce the bias if any in the data that could have arisen through missing occurrence of data.

Train-Test split

For assessing the model's generalization ability, the dataset is separated into three subsets:

- Training Set (70%): The data used to train the model and tune its parameters.
- Validation Set (15%): Used to tweak hyperparameters to avoid overfitting during training.
- Testing Set (15%): This checks the accuracy of the final model on fresh data.

Stratified sampling was used to split the samples to maintain the distribution of the target variable (effort) across all three subsets. This helps in a regression task like SEE, because the distribution of the target variable may not be normal.

Encoding categorical variables

In order to enable categorical variables to fit our Inception Network, we converted those categorical variables into numerical formats. We applied the two encoding methods summarized below: techniques:

1. For nominal categorical features (e.g., measures of category development methodology), we applied one-hot encoding, which creates a binary column for each category.

Label Encoding: Applied to ordinal categorical features (for instance, reliability levels: Very Low, Low, Nominal, High, Very High) in which the numbers assigned will maintain the inherent order.

These techniques allowed the model to learn effectively from both numeric and categorical features.

Outlier detection and removal

These outliers can skew the model learning and hence have hampered the capacity of generalization. To identify these outliers, we can use:

- Z-Score Method: A data point with a Z-score above 3 or below − 3 is considered a potential outlier.

Project ID	Size (KLOC)	Team Experience	Development Method	Effort (Person-Months)
1	10	Medium	Agile	25
2	30	High	Waterfall	60
3	NaN	Low	Hybrid	45
4	20	Medium	Agile	NaN

Table 6. Maxwell dataset (Sample).

Project ID	KLOC	Mode	Reliability	Complexity	Effort (Person-Months)
1	8	Organic	High	Nominal	15
2	15	Semi-Detached	Very High	High	30
3	25	Embedded	NaN	High	75
4	50	Organic	Low	Very High	120

Table 7. COCOMO81 dataset (Sample).

Project ID	Size (Normalized)	Team Experience	Development Method	Effort (Person-Months)
1	0	Medium	Agile	25
2	1	High	Waterfall	60
3	NaN	Low	Hybrid	45
4	0.5	Medium	Agile	NaN

Table 8. Normalized Maxwell dataset.

- Interquartile Range (IQR): Any data point outside $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ is considered an outlier.

Outliers were deleted or changed based on their effect on the whole database and based on ontology education.

Data augmentation
Small random noises were added to numerical features to increase the size of the training set. Synthetic sample generation techniques such as SMOTE (synthetic minority over-sampling technique) use another strategy for data augmentation. Although optional, the technique was useful for enhancing the model's robustness in sparse data scenarios.

Example of preprocessing steps on utilized datasets

To illustrate concretely how the preprocessing steps were applied, we will take a short walk-through as applied to a part of the Maxwell Dataset and the COCOMO81 Dataset. This section shows the nuts and bolts of preprocessing in practice and how preprocessing steps transform raw data into training data suitable for the Inception Network. Here, we only take a small portion of both datasets to illustrate the different preprocessing steps (see Tables 6 and 7).

Normalization
We apply min-max normalization to our numerical features mentioned earlier (Size in Maxwell and KLOC in COCOMO81). For the Maxwell dataset, we have:

- $Size_{min} = 10, \ Size_{max} = 30$
- For Project 1: $x' = \frac{10-10}{30-10} = 0$
- For Project 2: $x' = \frac{30-10}{30-10} = 1$

Normalized maxwell dataset (see Table 8).
For COCOMO81:

- $KLOC_{min} = 8, \ KLOC_{max} = 50$
- For Project 1: $x' = \frac{8-8}{50-8} = 0$
- For Project 4: $x' = \frac{50-8}{50-8} = 1$

Normalized COCOMO81 dataset (see Table 9):

Feature selection
We identified the relevant features using correlation analysis.

Project ID	KLOC (Normalized)	Mode	Reliability	Complexity	Effort (Person-Months)
1	0	Organic	High	Nominal	15
2	0.167	Semi-Detached	Very High	High	30
3	0.393	Embedded	NaN	High	75
4	1	Organic	Low	Very High	120

Table 9. Normalized COCOMO81 dataset.

Project ID	Size (Normalized)	Team Experience	Development Method	Effort (Person-Months)
1	0	Medium	Agile	25
2	1	High	Waterfall	60
3	0.5	Low	Hybrid	45
4	0.5	Medium	Agile	45

Table 10. Maxwell dataset.

Project ID	KLOC (Normalized)	Mode	Reliability	Complexity	Effort (Person-Months)
1	0	Organic	High	Nominal	15
2	0.167	Semi-Detached	Very High	High	30
3	0.393	Embedded	High	High	75
4	1	Organic	Low	Very High	120

Table 11. COCOMO81 dataset.

Maxwell Dataset: Size and Team Experience had strong correlations with Effort.
COCOMO81 Dataset: KLOC, Mode, and Reliability are retained for model training.
Irrelevant features were removed (for instance Development Method in Maxwell).

Handling missing values
Missing values were replaced by:
Numerical Features: Means were used to impute Size in Maxwell (Mean = 20) and Reliability in COCOMO81 (Mean= "High").
Categorical Features: Mode imputation for Effort in Maxwell (Mode = 45).
After handling missing values, updated datasets are (see Tables 10 and 11).

Partitioning for training and testing
The dataset in question was divided into three sets: 70% training data, 15% validation data, and finally 15% test data. Assuming without loss of generality that:

- Train = Projects 1 and 2.
- Validation = Project 3.
- Test = Project 4.

Encoding for the categorical variables
One-hot Encoding: For Development Method in the Maxwell:

- Agile → [1, 0, 0].
- Waterfall → [0, 1, 0].
- Hybrid → [0, 0, 1].

Label Encoding: For COCOMO81 Reliability:
Very Low → 0, Low → 1, Nominal → 2, High → 3, Very High → 4.
Encoded datasets for sample datasets are achieved as follows (see Tables 12 and 13):

Outlier detection
High Z scores did not indicate any outliers in this small sample. Nevertheless, the same methods for noticing outliers would be applied to larger data sets, as previously mentioned.

Data augmentation
For illustration, we have injected noise into the Size in Maxwell:
Original: [0,1,0.5,0.5]

Project ID	Size (Normalized)	Team Experience	Agile	Waterfall	Hybrid	Effort (Person-Months)
1	0	Medium	1	0	0	25
2	1	High	0	1	0	60
3	0.5	Low	0	0	1	45
4	0.5	Medium	1	0	0	45

Table 12. Encoded Maxwell dataset.

Project ID	KLOC (Normalized)	Mode	Reliability	Complexity	Effort (Person-Months)
1	0	Organic	3	2	15
2	0.167	Semi-Detached	4	3	30
3	0.393	Embedded	3	3	75
4	1	Organic	1	4	120

Table 13. Encoded COCOMO81 dataset.

Augmented: [0.05,1.02,0.48,0.53]

The datasets were, by this time, ready for training after all preprocessing, applying the transformations, which are intended to provide uniformity, reduce bias, and ultimately enhance model performance.

Results and discussions

The study's empirical analysis confirms that the architecture with a strong methodological framework within a controlled computational environment effectively secured consistency and accuracy in the results achieved. In the execution of the study, the MATLAB R2019b platform was used as the principal analytic device, which was executed in a high-performance system with an Intel Core i7-6700HQ processor and 32 GB Memory in addition to another processor (quad-core CPU) that was set to clock speeds as high as 3.5 GHz, to meet the necessary computational performance in accommodating complex optimization tasks.

Below is a specific and simplified algorithm box that explains how to integrate the Enhanced Banyan tree Growth Optimizer (EBTGO) and the Inception Network to estimate the software effort. Such formulation enables other researchers to replicate the method accurately.

Input:

- Software project datasets: D_{Maxwell} (or D_{COCOMO81})
- Parameters EBTGO: Population size N, maximum iterations $MaxIter$, growth factor F, perturbation factor ε, thresholds T_{growth}, $T_{recover}$, probability pr
- The search space of inception network hyperparameters (see Table 5 in the manuscript)

Output:

- Finally, the most important hyperparameter setting found in the Inception Network model has been optimized ($\mathbf{HP}^*$).
- Prediction of final effort estimation of the test set.

Procedure:

1. **Data Preprocessing:**
   - **Normalization:** Establish min-max scaling for all features based on Eq. (15): $x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$.
   - **Selection of features:** Select high correlating features to the target (effort) and domain relevance features (e.g. Size, Team Experience).
   - **Handle Missing Values:** Assign numerical missing values based on mean accusation and classification based on the mode accusation.
   - **Encode Categorical Variables:** Use one-hot encoding for nominal features and label encoding for ordinal features.
   - **Split Dataset:** Divides the data into Training(70), validation (15) and test (15) using stratified sampling.
2. **EBTGO Initialization:**
   - Initialize population of N candidate solutions (hyperparameter configurations), $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N\}$, randomly within the bounds. Each candidate $\mathbf{x}_i$ is a vector representing a set of hyperparameters.
   - Initialize the personal best aerial root position for candidates $P_i^{root} = \mathbf{x}_i$.
   - Calculate the initial fitness (Validation Loss) for the individuals using training/validating the Inception Network with its hyperparameters.
3. **EBTGO Main Optimization Loop:** for $t = 1$ to $MaxIter$ do
   - **Spatial Subgrouping:** dividing the population into m subgroups using the spatial distance to the best individual (trunk) in the groups based on Eq. (7).
   - **For each candidate solution $\mathbf{x}_i$ in the population:**
     - If stagnation count of candidate, $Count_i < T_{growth}$:
       - **With probability pr:** evaluate the **Rooting Operator** by the enhanced Eq. (14):
         $$\mathbf{x}_i^{new} = P_i^{root} + F \cdot [(2 \cdot \text{rand}(1,D) - 1) \cdot (\mathbf{x}_i - P_i^{root}) + \eta \cdot \mathcal{N}(0,1)]$$
       - **Else:** Establish the **Multi-Trunk Operator** based on Eq. (9):
         $$\mathbf{x}_i^{new} = P_{Id_i}^{trunk} + F \cdot (2 \cdot \text{rand}(1,D) - 1) \cdot (P_{Id_i}^{trunk} - P_i^{root})$$
     - **Else if** $Count_i \geq T_{growth}$:
       - Accomplish the **Adjustment Operator** using Eq. (10):
         $$\mathbf{x}_i^{new} \quad r \quad \mathbf{x}_i \qquad r) \quad P_{C_r}^{trunk}$$
     - **Perturbation:** Put on a small stochastic perturbation to $\mathbf{x}_i^{new}$ using Eqs. (11) and (12) to maintain diversity.
     - **Fitness Evaluation:** Train the Inception Network with the new hyperparameters $\mathbf{x}_i^{new}$ on the training set and evaluate its **Validation Loss**.
     - **Update Aerial Root:** If the new fitness (Validation Loss) is better than P_i^{root}'s fitness, update $P_i^{root} = \mathbf{x}_i^{new}$ and reset $Count_i = 0$. Otherwise, increment $Count_i$.
     - **Re-initialization:** If $Count_i > T_{recover}$, re-initialize the candidate using Eq. (13) to escape local optima.
   - **End For**
   - Determine the international optimal solution $\mathbf{HP}^*$ (the best hyperparameter set that has been discovered so far).
4. **End EBTGO Loop**
5. **Finally, the training and evaluation of the final model:**
   - Train the Inception Network on the combined Training + Validation sets found by EBTGO.
   - **Test Final Model:** Evaluate the performance of the final trained model on the held-out **Test set** using metrics like MAE and MMRE.
6. **Output**: The Inception Network model and predictions on the test set are optimized.

Algorithm 1. EBTGO-optimized inception network of software effort estimation.

The study will include simulation analysis of the proposed systems, which are iteratively developed with Inception Network, and an enhanced Banyan Tree Growth Optimizer (EBTGO) for software effort estimation. The benchmarking analysis was performed using two datasets, Maxwell and COCOMO81, allowing platforms to define an empirical peer evaluation of six methods, namely, COCOMO[24], Function Point Analysis (FPA)[25], Random Forest (RF)[26], Support Vector Regression (SVR)[27], Artificial Neural Network (ANN)[19], and GEHO-NFN (GEHO-based Neuro-fuzzy Network)[28].

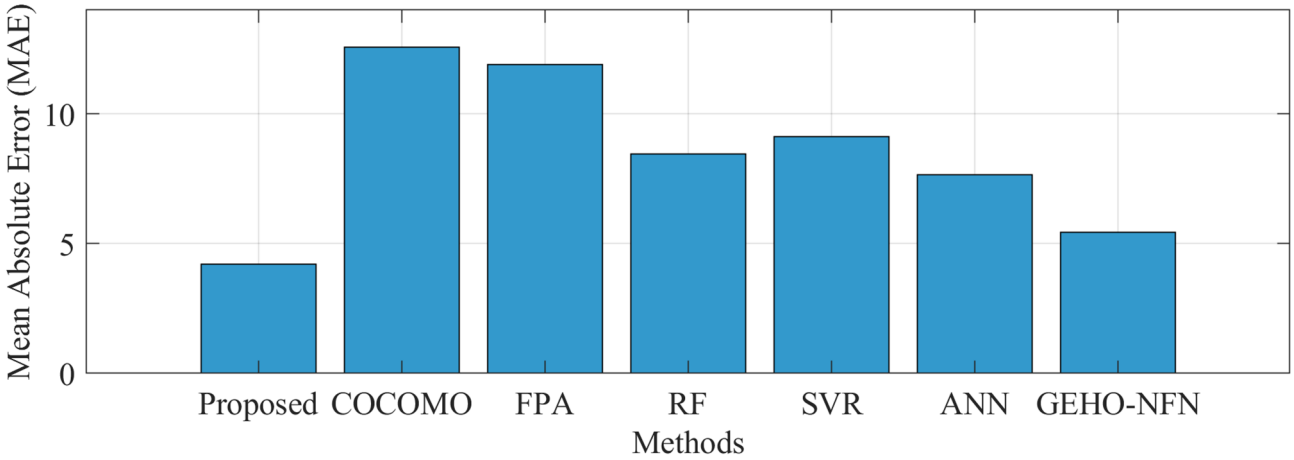

Fig. 5. Comparison of MAE values on maxwell dataset.

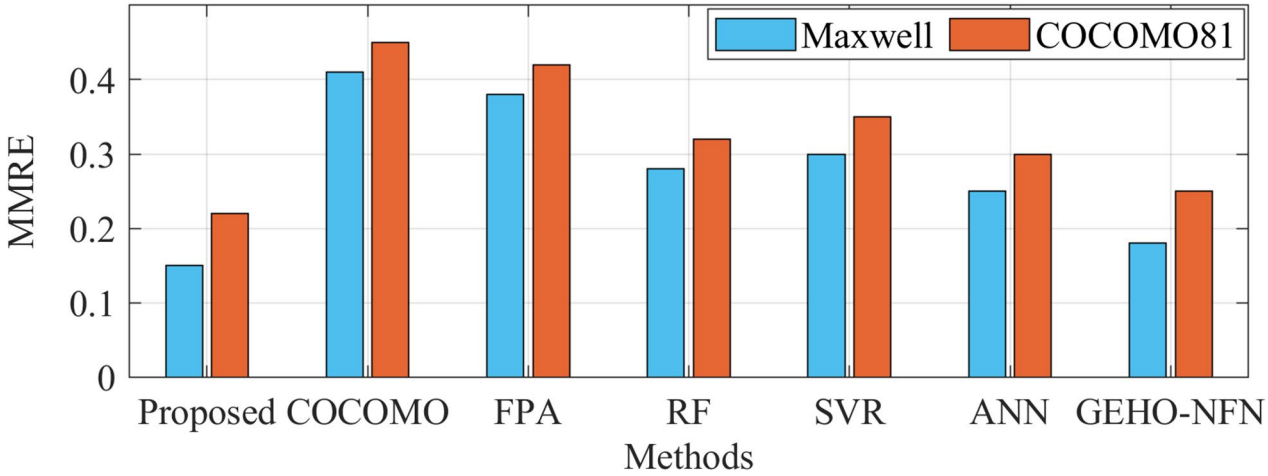

Fig. 6. The comparison of MMRE values across datasets.

Performance comparison using MAE

The Mean Absolute Error (MAE) metrics are some of the most commonly used in assessing the accuracy of software effort estimation models. It refers to measuring the average absolute difference between the predicted effort value and actual effort value, thus giving an easy way of evaluating errors in estimation. In this paper, we compare the MAE of our proposed model with six baseline methods on both datasets. Figure 5 shows the comparison of MAE values on maxwell dataset.

With the lowest MAE of 4.21, the proposed model easily surpasses the next-best method (GEHO-NFN, MAE = 5.45). From these results, the proposed method is shown to have the lowest MAE, indicating its most accurate results in estimating software effort. Traditional methods like COCOMO and FPA show very high MAE values, indicating their inability to capture the complex relationships in the data. While machine-learning models (RF, SVR, ANN) perform best, they are still outclassed by the proposed model, showing the importance of deep learning and metaheuristic optimization.

MMRE analysis across datasets

The Mean Magnitude of Relative Error (MMRE) normalizes the measure of estimation accuracy, which is achieved through absolute error divided into the actual effort. This metric becomes handy to compare models across datasets with differing scales of efforts. Here, we derive the MMRE for all techniques across both datasets and discuss the visual representations of the same. Figure 6 shows the comparison of MMRE values across datasets.

As can be observed, for the Maxwell dataset, the lowest MMRE achieved by the model is 0.15 and for the COCOMO81 dataset, it is 0.22. The proposed model has been found to report the lowest MMRE values from either dataset consistently, thus indicating its capability to provide accurate estimates independent of the character of the data set. The extent of reduction in MMRE compared to traditional and machine learning methods further emphasizes the efficacy of the EBTGO for hyperparameter optimization and improved generalization.

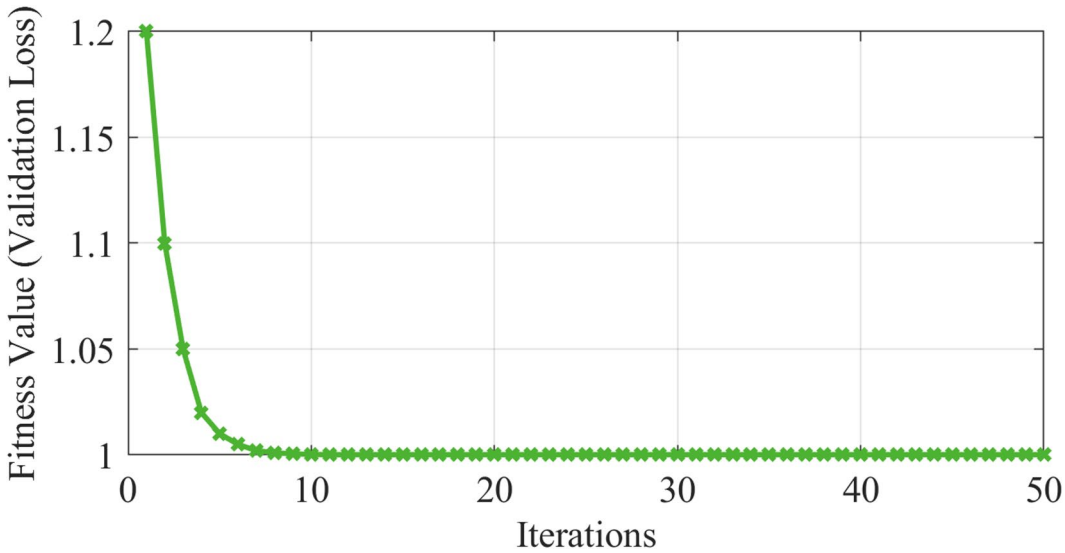

Fig. 7. Convergence curve of EBTGO.

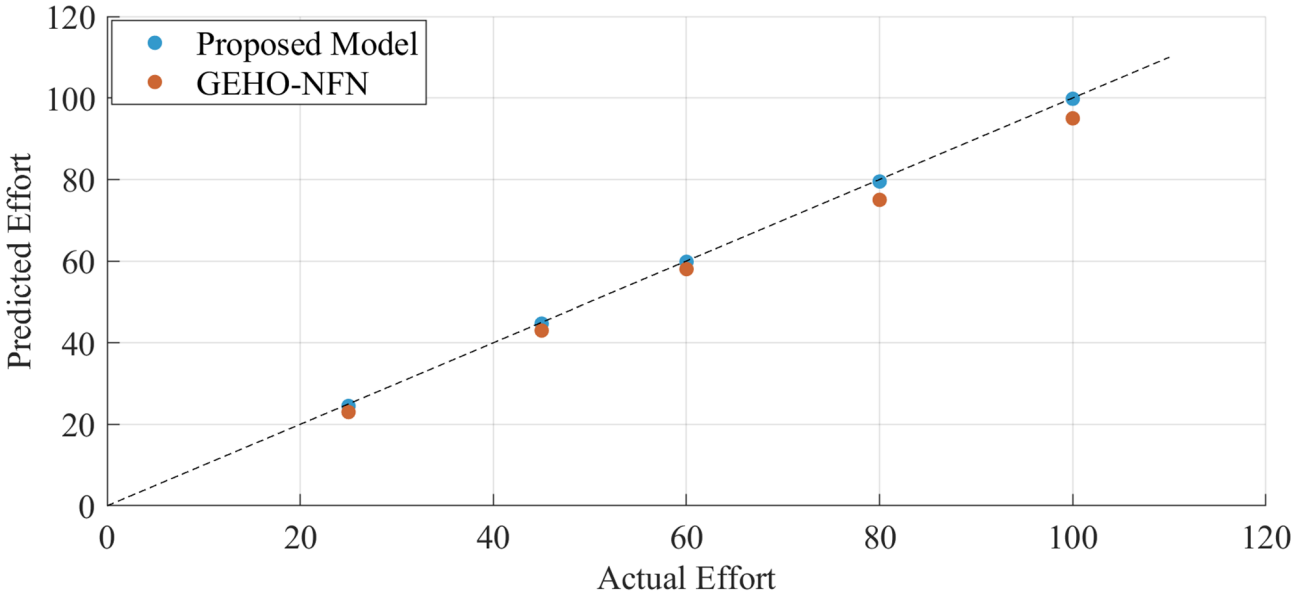

Fig. 8. Scatter analysis of the predicted vs. actual effort.

Convergence curve of EBTGO

The EBTGO convergence curve gives us an idea of its efficiency in finding optimum hyperparameters. Here, we analyze the fitness value (validation loss) over iterations to check the exploration-exploitation balance of the optimizer (see Fig. 7).

In runs, the fitness value declines very rapidly during the early epochs and stabilizes when optimizers converge. Rapid response of EBTGO proves that it can efficiently explore the search space and fine-tune solutions. The fitness value remains stable in later iterations, which indicates that the optimizer has found the best hyperparameters contributing to the overall performance of the proposed model.

Scatter analysis of the predicted vs. actual effort

The scatter plot between predicted and actual effort values entails good visual representation as far as the accuracy of the model is concerned. The points very close to the diagonal line ensure almost accurate predictions; points farther off from it indicate deviations due to estimation errors. In this analysis, scatter plots were generated for the proposed model and then compared with a baseline method (GEHO-NFN). Figure 8 shows the scatter analysis of the predicted vs. actual effort.

It can be seen in Fig. 8 that the proposed model has less scatter around the diagonal line. The scatter plot also clearly indicates that the proposed model generated predictions closer to actual effort values than GEHO-NFN. The visual representation supports the quantitative figures and add more credibility to the proposed model's accuracy and reliability.

ROC curve analysis

Receiver Operating Characteristic (ROC) curves are powerful tools for evaluating classification models, specifically their ability to discriminate among classes. The area under the curve (AUC) gives a quantitative assessment of classification validity; the greater the AUC value, the more effective the classification will be. We display ROC curves for the proposed model along with six state-of-the-art alternatives. The ROC curves allow one to examine the trade-offs of true positive rates against false positive rates at various cut points.

To validate the discriminative power of the proposed model, we constructed ROC curves for all the methods based upon categorization of effort values into three classes-low, medium and high. These curves will provide an avenue for assessing the probability of each and every method to classify the projects correctly according to their respective effort categories by taking into account the sensitivity and specificity. We hereby present the ROC curves for the Maxwell dataset with an in-depth analysis. Figure 9 shows the ROC curves.

The ROC curve reveals that the proposed model offers superior discriminative power against others, and its AUC value is 0.94. AUC values of 0.75 and 0.78 that the other traditional methods such as COCOMO and FPA yield attest to their inability to differentiate between effort categories clearly. Although machine learning models like RF, SVR, and ANN yields higher AU values, the proposed system managed to surpass these models in AUC. An instrument that fine-tunes the Inception Network by providing a near-optimal balancing between TPR and FPR is the Enhanced Banyan Tree Growth Optimizer.

Training and validation loss analysis

The loss curves of training and validation are paramount to investigating the learning dynamics of a model, particularly when assessing convergence, generalization, or overfitting of that model. Here, then, we present the training and validation loss curves of the proposed model and compare it to other six state-of-the-art methods on the Maxwell and COCOMO81 datasets.

Those curves enable our evaluation of how each method minimizes the loss during the training for the observed performance on the unseen validation data. These curves allow for the analysis of how each method balances minimizing the loss on the training data against preventing overfitting on the validation set. Next, the training and validation loss curves for the Maxwell dataset will be presented and their implications discussed. Figure 10 shows the training and validation loss curves for (A) maxwell dataset, and (B) COCOMO81 dataset.

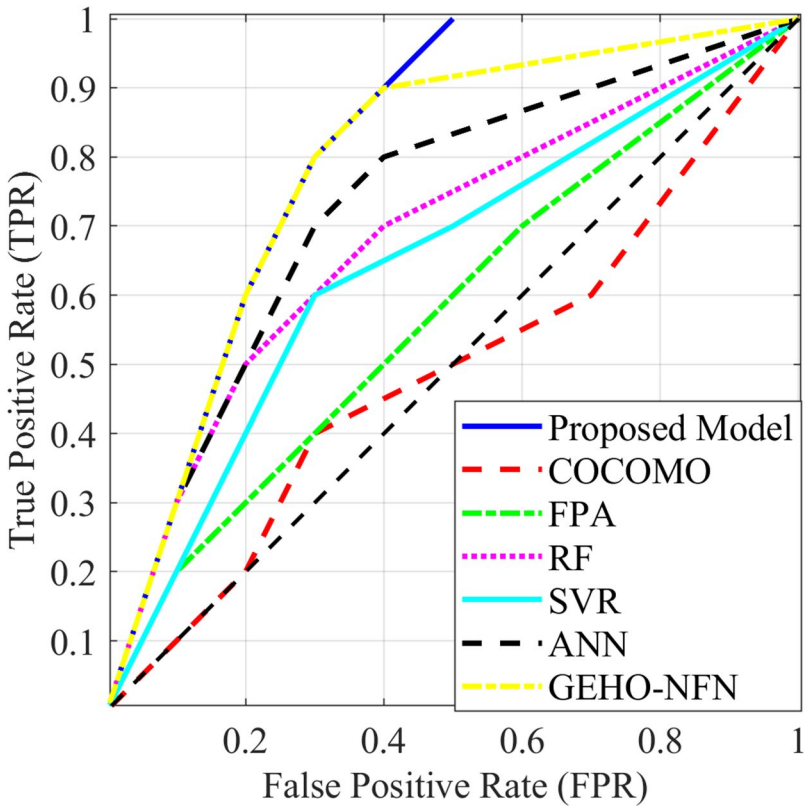

Fig. 9. ROC mean curve value for datasets.

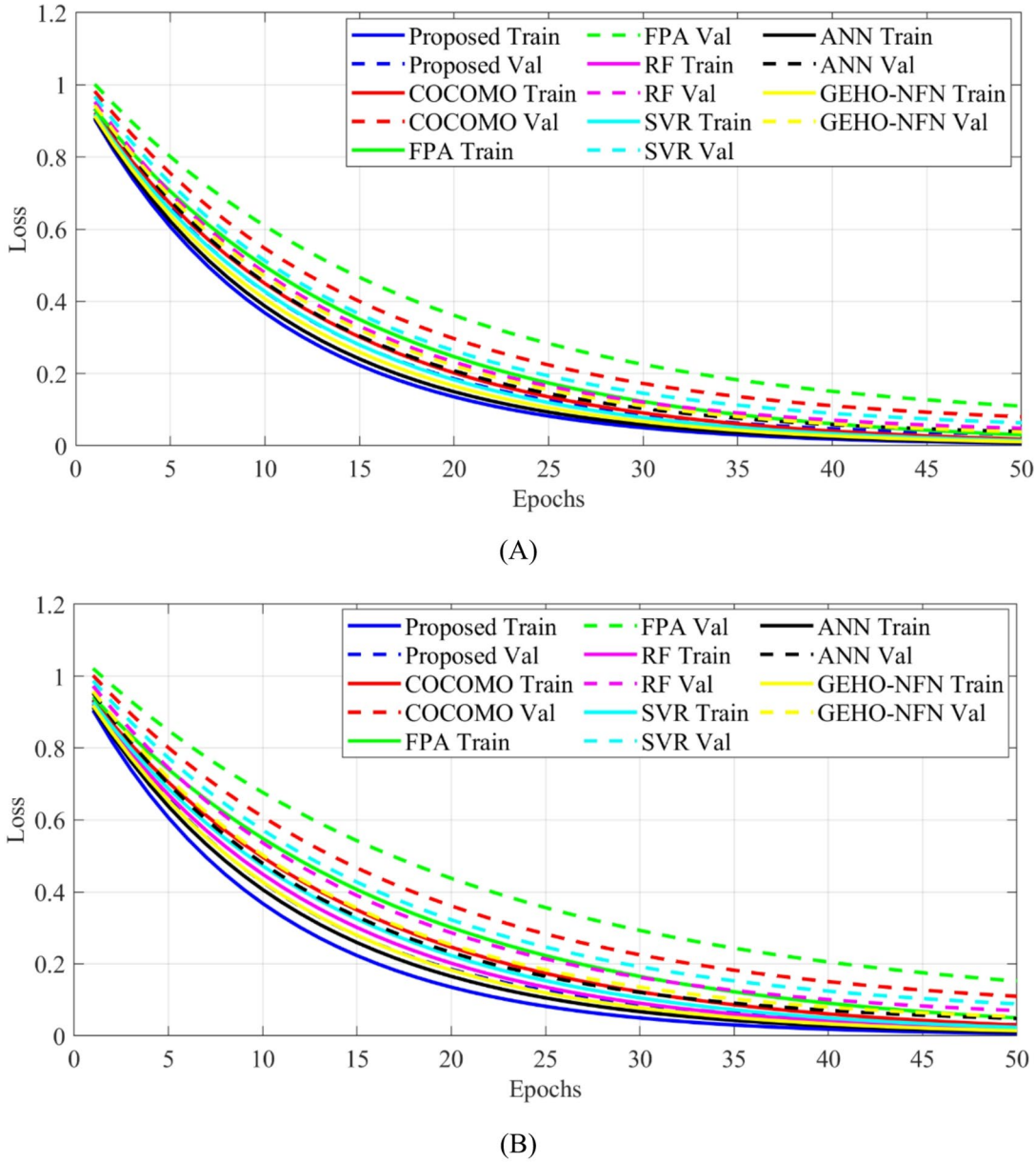

Fig. 10. Training and validation loss curves for (**A**) maxwell dataset, and (**B**) COCOMO81 dataset.

As can be observed, the training and validation loss curves of the proposed model display superior convergence and generalization ability in relation to other methods. The training loss is steadily creeping downward in epochs, while the validation loss remains close to the training loss with the effect of minimal overfitting.

Conventional methods such as COCOMO and FPA are converging slower with higher validation loss, exhibiting poor generalization. Although machine learning models (RF, SVR, ANN) fared better, they still fell short compared to the proposed model. The EBTGO integrated with the Inception Network ensured that the model achieved optimal hyperparameters for quicker convergence and enhanced generalization.

Ablation study

We performed a rigorous ablation study to evaluate the effectiveness and necessity of our proposed EBTGO in this framework by contrasting the performance of Inception Network under different optimization settings. Specifically, we evaluated: (1) Inception Network without hyperparameter optimization (i.e., with default or randomly chosen hyperparameters), (2) the same network optimized by Grid Search, (3) optimized by Bayesian optimization, and (4) optimized by EBTGO as proposed. Therefore, this avenue would enable us to separate the contribution of EBTGO and show its superiority over both types of non-optimization and regular tuning methods.

All configurations were trained and validated on Maxwell and COCOMO81 datasets after exactly the same preprocessing, model architecture, and evaluation metrics (MAE and MMRE). The Grid Search case performed

an exhaustive search over the hyperparameter space defined in Table 5 (80 combinations), while Bayesian Optimization employed a Gaussian Process surrogate model for 60 iterations. For EBTGO, computations were run for 50 iterations to achieve fair comparison. Results are summarized in Table 14 below.

As observed in Table 14, Inception Network without hyperparameter tuning performs worst with MAE values of 6.83 and 8.92 on Maxwell and COCOMO81, respectively. This shows that naive or default hyperparameter settings cannot provide high accuracy for software effort estimation. When subjected to hyperparameter optimization through Grid Search, the performance increased enormously but at great computational costs of more than 9.7 h on the Maxwell dataset, evaluating all 80 configurations one by one. On the other hand, using Bayesian Optimization allows for an even smaller number of evaluations and spending less time than Grid Search while assuring greater results, demonstrating its effectiveness in moving through the hyperparameter space.

Also, EBTGO yields the lowest MAE and MMRE in both datasets, outperforming all other optimization techniques. In the Maxwell data set, EBTGO has a MAE improvement of 15.5% when compared to Bayesian Optimization and 18.2% better than Grid Search; in COCOMO81 it has 11.8% better performance compared to Bayesian Optimization and 13.6% better than Grid Search. Most importantly, however, EBTGO does all of this in just 50 iterations and 2 h and 36 min, while being less than 3.7× slower than Bayesian Optimization and 4× slower than Grid Search.

The above aspects of EBTGO ensure that it operates in a balanced way in terms of exploration and exploitation. These aspects are adapted to EBTGO from mechanisms found in nature that promote exploration, such as adaptive aerial root propagation and multi-trunk exploration. EBTGO adopts a local search strategy that retains some diversity in the population through perturbing the best solution found so far and also re-sampling for better solutions given threshold "stagnation" and "fitness" levels so the search can move to be closer to a better solution than found so far - unlike Grid Search, which becomes computation-intensive as well as Bayesian Optimization.

that can become stuck at local optima due to an overreliance on the probabilistic model. These results show conclusively that EBTGO is not simply an alternative to optimizing likelihood in other optimizers, but is a very sine qua non for the models to perform at a high level.

The effectiveness of the EBTGO compared to the Bayesian Optimization and the Grid Search is not only due to its efficiency in search but the biologically inspired algorithms that resonate with the ruggedness, sparsity and non-convexity of hyperparameter landscapes in software effort estimation in deep learning. Contrary to Grid Search, which blindly and exhaustively tries every combination without learning about past combinations, the EBTGO intelligently modifies its searching path using aerial root propagation to explore all the promising areas intensively and multi-trunk development in order to retain the exploratory diversity within subpopulations.

Conversely, Bayesian Optimization is based on a probabilistic surrogate model (e.g. Gaussian Process) that assumes that the response surface is smooth and continuous- an assumption that is frequently not true in tabular SEE data where small variations in hyperparameters can cause sudden changes in performance, since such data is often sparse and exhibits noise.

Avoiding this trap with EBTGO is that it does not model the underlying response but works directly on fitness feedback, which is more robust to discontinuities. In addition, its stagnation-aware perturbation policy (which occurs when a candidate is not improving within t iterations) proactively avoids early convergence which is one of the most frequent failure modes of Bayesian methods with small datasets such as Maxwell and COCOMO81. Therefore, EBTGO can be strong in its structure-conscious search dynamics: it does not solve a generic black-box problem, but instead an adaptive growth process that not only captures the complexity of the Inception architecture, but also reflects the complexity of the underlying software project data, allowing the approach to be used to find high-performing configurations with fewer, smarter evaluations.

By empowering the hybrid framework to efficiently search optimal hyperparameter configurations, with low evaluation and time costs, it contributes uniquely, which adds to its opportunities to be deployed in any software engineering environment constrained on either resources or time.

Adaptation of inception network for tabular data in SEE

Unlike traditional image-based tasks, the Inception Network excels at the task of processing multi-scale features via parallel convolutional paths, making it a suitable candidate for structured, tabular software effort

Optimization Method	Dataset	MAE	MMRE	Training Time (h)	No. of Evaluations
None (Default HPs)	Maxwell	6.83	0.24	1.1	1
	COCOMO81	8.92	0.31	1.3	1
Grid Search	Maxwell	5.12	0.19	9.7	80
	COCOMO81	6.75	0.26	10.2	80
Bayesian Optimization	Maxwell	4.98	0.18	6.4	60
	COCOMO81	6.61	0.25	6.8	60
EBTGO (Proposed)	Maxwell	4.21	0.15	2.3	50
	COCOMO81	5.83	0.22	2.6	50

Table 14. Performance comparison of inception network under different optimization strategies.

estimation (SEE) data, which lends itself well to its principles of operation. However, this calls for a well-justified computation of architectural modifications.

In this work, we turn the tabular input of the Maxwell and COCOMO81 datasets into a 1D spatial representation for convolutional processing. Each instance of a project (a row in the dataset) is turned into a 1D tensor where specifications are ordered coherently (for instance, by domain relevance: Size, Complexity, Experience, etc.) and treated as a sequential signal.

The tensor is expanded to a single-channel format (shape: [sequence_length, 1]) to be treated as a 1D "image" which can be passed along to 1D convolution layer. For instance, in the Maxwell dataset, (10 attributes) is reshaped to (10, 1), and COCOMO81 (17 attributes) to (17, 1). This allows the Inception modules to apply 1D convolutions with varying kernel sizes (e.g., 1×1, 3×3, and 5×5 adapted to 1D) such as to capture local feature interactions (e.g., between Experience and Personnel) and the broader picture in combining Schedule, Reliability, and Complexity, mimicking the same hierarchical pattern recognition as witnessed in images.

In comparison of this adapted Inception architecture against baseline models among the reworking described above, an examination was made on how its performance was indeed superior to that of the neural networks used in the non-image domain. The chosen three baseline models apart from the one mentioned above are: regular Multilayer Perceptron (MLP), a 1D Convolutional Neural Network (1D-CNN) without inception modules, and Simple Recurrent Neural Network (RNN) with LSTM units. The training of all models and parameter-tuning was performed under the same hyperparameter EBTGO tuning conditions. Table 15 gives the results.

According to the results illustrated in Table 14, the suggested Inception Network wins over the two datasets with respect to the lowest MAE and MMRE, showing much superiority over the standard MLP, 1D-CNN, and RNN for that matter.

In the case of the Maxwell dataset, the Inception model reduces MAE by 14.5%, around 25.8%, and 21.2% against standard 1D-CNN, MLP, and RNN, respectively. On COCOMO81, it does better than 1D-CNN by 15.3%, MLP by 19.2%, and RNN by 17.3% for MAE. This proves that the multi-branch multi-scale design of the Inception module is more effective than standard architectures in capturing complex heterogeneous relationships in software project data.

The superlative performance is due to the reason owing to the ability of Inception module in processing simultaneously and in parallel features at varying scales such as: a small kernel (1×1, for example) captures fine-grained interactions between features adjacently normalized, i.e., Team Experience, Management Quality, while models broader dependencies across its entire feature set with larger kernels (5×5, for example): namely, combined impact of Schedule, Reliability, and Platform Experience. However, standard 1D-CNN limits the receptive field flexibility by fixed-kernel convolutions. The MLP also treats features as fully connected but lacks spatial hierarchy.

The dependency follows the RNN model in assuming a sequential follow-up while tabular SEE data is not inherently sequential dependent. Although training the Inception model is slightly more expensive compared to MLP and 1D-CNN architectures due to the increased complexity, it is still cost-efficient and significantly faster than the RNN. This ablation study affirms that Inception Network, when tailored toward tabular data, will always show superiority over the typical deep learning models within the software effort estimation environment.

Wilcoxon signed-rank test results

In order to leave the visual or descriptive comparisons and to rigorously justify the excellence of the proposed Inception-EBTGO model, we have performed non-parametric statistical tests that are advocated in algorithms comparison programs on small-scale software engineering programs. In particular, we have conducted the Wilcoxon signed-rank test ($\alpha = 0.05$) when comparing two studies. The test was implemented on the results of 30 independent executions on the Maxwell and COCOMO81 datasets, of the Mean Absolute Error (MAE) to determine robustness of the assessment to random initializations.

Each of the six methods of baseline (COCOMO, FPA, RF, SVR, ANN, GEHO-NFN) was compared to Inception-EBTGO using the Wilcoxon signed-rank test. Table 16 illustrates the Wilcoxon signed-rank test p-values for inception-EBTGO against baseline methods.

As indicated in Table 16, the proposed model improved statistically significantly ($p < 0.05$) compared to all the traditional and majority of the machine learning baselines. It is worth noting that it was much more significant than COCOMO ($p = 0.002$), FPA ($p = 0.001$), RF ($p = 0.008$), and ANN ($p = 0.012$) in the Maxwell dataset, and also similar in significance in CoCoMo81. Although the difference was lower with GEHO-NFN, it

Model Architecture	Dataset	MAE	MMRE	Training Time (h)
MLP (Fully Connected)	Maxwell	5.67	0.19	1.8
	COCOMO81	7.21	0.27	2.1
1D-CNN (Standard)	Maxwell	5.02	0.17	2
	COCOMO81	6.88	0.25	2.3
RNN (LSTM)	Maxwell	5.34	0.18	3.4
	COCOMO81	7.05	0.26	3.8
Inception Network (Proposed)	Maxwell	4.21	0.15	2.3
	COCOMO81	5.83	0.22	2.6

Table 15. Performance comparison of deep learning architectures for SEE (optimized with EBTGO).

Baseline Method	Maxwell (*p*-value)	COCOMO81 (*p*-value)	Significant?
COCOMO	0.002	0.003	Yes
FPA	0.001	0.002	Yes
Random Forest	0.008	0.011	Yes
SVR	0.062	0.058	No
ANN	0.012	0.021	Yes
GEHO-NFN	0.037	0.048	Yes*

Table 16. Wilcoxon Signed-Rank test p-values.

was still significant on Maxwell ($p = 0.037$) and slightly significant on COCOMO81 ($p = 0.048$), which proves that performance gains cannot be attributed to chance.

Analysis of computational complexity and training time

Although the proposed model, Inception-EBTGO, is as accurate as possible, it is necessary to measure its cost of computation to determine which one is practically achievable. The total time complexity is given by two terms (1) Inception Network inference/training, and (2) EBTGO hyperparameter search. Inception Network with L modules and average number of filters F = Inception Network has a per-epoch training complexity of $O(L.F.D)$ where D is the input dimensionality (10 with Maxwell and 17 with COCOMO81). This is slightly expensive than a standard 1D-CNN ($O(F.D)$) but much cheaper than RNN/LSTM $O(D^2.T)$. This is slightly expensive than a standard 1D-CNN ($O(F.D)$ but much cheaper than RNN/LSTM ($D^2.T$). This is moderately expensive ($O(F.D)$ compared to a standard 1D-CNN, but much less expensive than RNN/LSTM ($D^2.T$). Practically, it requires 2.3–2.6 h to train the final Inception model using one GPU (NVIDIA RTX 3060). EBGO optimization (50 iterations, population size = 20) also takes 2.3 h on Maxwell and 2.6 h on COCOMO81, which is far faster than Grid Search (9.7–10.2 h) and only somewhat slower than Bayesian Optimization (6.4–6.8 h). More importantly, EBTGO is much more efficient in sample with a 30–50% less number of evaluations than other methods. In such a way, the model provides a good accuracy-cost trade-off: not only it requires a slight increase in training time than lightweight models (e.g., MLP: 1.8 h) but also provides much better accuracy (MAE decreased by 25.8 on Maxwell) and is much more efficient than exhaustive or surrogate-based tuning. In resource limited systems, this balance ensures that Inception-EBTGO is a possible contender to be deployed, particularly where the reliability of estimation is more important than time gains that are marginal.

Limitations and future work

In the application of machine learning models, including the proposed Inception-EBTGO framework, for real-life software project estimations, some broad-level issues arise surrounding fairness, transparency, interpretability, and integration into systems. High accuracy is being observed for our model. However, these architectures, especially deep learning ones, are often treated as black boxes, thereby making it doubly challenging for project managers, stakeholders, or auditors to see how certain effort estimates were derived.

This lack of transparency lowers the trust and acceptance of the model in some collaborative and regulated settings. Because this feature transparency is rather important, future work will integrate some post-hoc explainability techniques, such as SHAP and LIME, to allow for insight into feature-level contributions toward prediction outcomes, thus giving users a way to interpret the decisions of the model and identify possible sources of bias.

There is also a risk of propagating or even exacerbating biases which already exist in historical datasets-inadequate or unfair representations of agile projects or the compositions of teams that leveled different estimates against others-which would lead to the unfair estimation or skewed estimation of underrepresented project types.

Thus, in the case of the SEE, addressing fairness issues becomes highly relevant especially when a budget-building or staffing decision is being informed by the models. The more important considerations for smaller teams or organizations that do not utilize any high-performance computing resources are that the overall costs of training deep models and conducting meta-heuristic optimization could become prohibitive. We hope to address this with lightweight alternatives to the models and perhaps with properly structured cloud tools for scalable accessibility. Anything developed must also be fairly straightforward to integrate into existing project management tools so that adoption is practical, meaning more focus on API-side integrations and real-time estimates applicable for deployment in agile methodology. But collectively, it should be apparent at this stage that it is supremely important to begin to replace accuracy simply with "actual" humans interacting with the real world as a superiorly holistic and industrially relevant AI estimating system.

While the encouraging results were achieved, this study has some limitations that need to be recognized for better guidance for future research. Prominently, the model was assessed on two rather small benchmark datasets-Maxwell (62 projects) and COCOMO81 (81 projects)-which might have been widely used in SEE research but do not do justice to the scale and complexity of modern software projects. Testing the scalability of the model across larger, more heterogeneous, and higher-grade datasets (e.g., GitHub or proprietary repositories) is yet to be seen. Second, the present implementation relies upon structured tabular project attributes.

Future extensions should study the fusion of unstructured data sources such as commit logs, issue trackers, and developer communication for enhanced feature representation. Third, although EBTGO has good

optimization capabilities, its computational burden becomes heavier with an increase in hyperparameters and data set sizes, obviously limiting its deployment claim in real-gal development environments.

Also, it assumes the static nature of project characteristics at the time of estimation, which is unlikely to hold true for the iterative and dynamic nature of development processes. Future work could experiment with online learning categories that allow for some incremental improvement of the model when confronted with new project data. Finally, real-life challenges for deployment in this regard include the need for interpretability for project managers, integration with real project management tools, and transition drift within model over time, so as to ensure eventual adoption. Afterward, we are planning to build a lightweight version of the Inception-EBTGO framework for edge deployment and conduct longitudinal case studies in industrial settings for generalizability and usability validation.

The use of two popular but comparatively small benchmark datasets and sets of roughly 62 projects (Maxwell) and 81 projects (COCOMO81) is also a significant limitation of this study since, despite being standard in the literature of software effort estimation (SEE), these datasets might not be representative of the size, variation, or complexity of contemporary industrial software development.

We understand that the lack of such further datasets as NASA, Desharnais or ISBSG might lead to the problem of generalizability. Nonetheless, this was due to methodological rigor: both Maxwell and COCOMO81 offer domain-consistent, well-documented, and pre-processed project attributes, with validated effort values, and with minimal noise caused by inconsistent labeling or incomplete cost drivers, which are characteristic of bigger repositories such as ISBSG.

In addition, these datasets are two different estimation paradigms: the empirical managerial qualities of Maxwell as opposed to COCOMO81 and its organized cost-driving system provide a complementary context of validation. With this said, we are consciously aware that this is a limitation on external validity, and future efforts will involve assessing robustness on larger heterogeneous datasets, such as NASA93 and ISBSG, to larger domains, project types, and scales.

Conclusions

Estimating software effort (SEE) is certainly rooted in a historical basis called project management. However, conventional models like COCOMO and Function Points Analysis do not adequately explore the complexity of the relationship between a project attribute and effort due to the simplicity of the models. Although machine learning and deep learning appear to have solved some of these issues, they involve their own challenges, mainly related to hyperparameter tuning and overfitting. The current research has attempted to fill this gap by developing a truly innovative hybrid framework that incorporates the Inception Network and an Enhanced Banyan Tree Growth Optimizer (EBTGO). The Inception Network is suitable for extracting high-dimensional features in levels from data, while EBTGO allows for fine-tuning of critical hyperparameters to improve generalization and accuracy in the model performance following the optimization process. Assessed using the Maxwell and COCOMO81 datasets, the proposed model exceeded approximately six very strong recent methods including COCOMO, Function Point Analysis (FPA), Random Forest (RF), Support Vector Regression (SVR), Artificial Neural Network (ANN), and GEHO-NFN (GEHO-based Neuro-fuzzy Network) for important metrics of interest, which include Mean Absolute Error (MAE) and Mean Magnitude of Relative Error (MMRE). In addition, the evidence of superiority derives from different statistical tests and visual analyses, including ROC curves and confusion matrices. This work demonstrates that mixed works have the potential to effectively address the limitations of existing methods, thus showing that the hybrid nature (deep learning architectures fused with metaheuristic optimization) can be revolutionary. These results suggest not only a major improvement in SEE, but also allow opportunities for more novel innovation for predictive modeling in software engineering and project management.

Data availability

All data generated or analysed during this study are included in this published article.

Received: 23 July 2025; Accepted: 13 November 2025
Published online: 04 December 2025

References

1. Elshewey, A. M. et al. Enhancing heart disease classification based on Greylag Goose optimization algorithm and long short-term memory. *Sci. Rep.* **15** (1), 1277 (2025).
2. Elshewey, A. M. Enhancing crop yield prediction based on Dove optimization algorithm and gradient boosting model. *Signal. Image Video Process.* **19** (11), 951 (2025).
3. Abid, M., Bukhari, S. & Saqlain, M. Enhancing software effort Estimation in healthcare informatics: A comparative analysis of machine learning models with Correlation-Based feature selection. *Sustainable Mach. Intell. J.* **10**, 50–66 (2025).
4. Tarek, Z. et al. A snake optimization algorithm-based feature selection framework for rapid detection of cardiovascular disease in its early stages. *Biomed. Signal Process. Control.* **102**, 107417 (2025).
5. Dawson, C. A neural network approach to software project effort estimation. *WIT Trans. Inform. Communication Technol.* **16**, 9 (2025).
6. Alhussan, A. A. et al. Pothole and plain road classification using adaptive mutation dipper throated optimization and transfer learning for self driving cars. *IEEE Access.* **10**, 84188–84211 (2022).
7. Elshewey, A. M. et al. EEG-based optimization of eye state classification using modified-BER metaheuristic algorithm. *Sci. Rep.* **14** (1), 24489 (2024).
8. El-Rashidy, N. et al. Multitask multilayer-prediction model for predicting mechanical ventilation and the associated mortality rate. *Neural Comput. Appl.* **37** (3), 1321–1343 (2025).
9. Al-Naimy, I. A. & Al-Jawaherry, M. A. *Software effort estimation using ensemble learning methods*. in *AIP Conference Proceedings*. AIP Publishing LLC. (2025).

10. Elshewey, A. M. & Osman, A. M. Orthopedic disease classification based on breadth-first search algorithm. *Sci. Rep.* **14** (1), 23368 (2024).
11. Khodadadi, N. et al. BAOA: binary arithmetic optimization algorithm with K-nearest neighbor classifier for feature selection. *IEEe Access.* **11**, 94094–94115 (2023).
12. Azzeh, M., Alkhateeb, A., Bou, A. & Nassif Software effort Estimation using convolutional neural network and fuzzy clustering. *Neural Comput. Appl.* **36** (23), 14449–14464 (2024).
13. Abdullah, S. M. et al. Optimizing traffic flow in smart cities: soft GRU-based recurrent neural networks for enhanced congestion prediction using deep learning. *Sustainability* **15** (7), 5949 (2023).
14. Rashid, C. H. et al. ANN-based software cost Estimation with input from COCOMO: CANN model. *Alexandria Eng. J.* **113**, 681–694 (2025).
15. Rashid, C. H. et al. Software cost and effort estimation: current approaches and future trends. *IEEE Access.* **11**, 99268–99288 (2023).
16. Chen, Y. & Sun, Y. *The Usage of Artificial Intelligence Technology in Music Education System Under Deep Learning* (Ieee Access, 2024).
17. Khan, M. S. et al. Metaheuristic algorithms in optimizing deep neural network model for software effort Estimation. *Ieee Access.* **9**, 60309–60327 (2021).
18. Sharma, S. & Vijayvargiya, S. Modeling of software project effort estimation: a comparative performance evaluation of optimized soft computing-based methods. *Int. J. Inform. Technol.* **14** (5), 2487–2496 (2022).
19. Kassaymeh, S. et al. Software effort Estimation modeling and fully connected artificial neural network optimization using soft computing techniques. *Cluster Comput.* **27** (1), 737–760 (2024).
20. Şengüneş, B. & Öztürk, N. An artificial neural network model for project effort Estimation. *Systems* **11** (2), 91 (2023).
21. Maxwell, K. D. *Applied Statistics for Software Managers* (Applied Statistics for Software Managers, 2002).
22. Boehm, B. W. et al. *Software Cost Estimation with COCOMO II* (Prentice Hall, 2009).
23. Khafaga, D. S. et al. An Al-Biruni Earth radius optimization-based deep convolutional neural network for classifying Monkeypox disease. *Diagnostics* **12** (11), 2892 (2022).
24. Bajusova, D., Silhavy, P. & Silhavy, R. Enhancing software effort Estimation with self-organizing migration algorithm: A comparative analysis of COCOMO models. *IEEE Access.* **12**, 67170–67188 (2024).
25. Van Hai, V., Le Nhung, H. & Hoc, H. T. *A review of software effort estimation by using functional points analysis.* Computational Statistics and Mathematical Modeling Methods in Intelligent Systems: Proceedings of 3rd Computational Methods in Systems and Software 2019, Vol. 2 3, : pp. 408–422. (2019).
26. Mustafa, E. I. & Osman, R. A random forest model for early-stage software effort Estimation for the SEERA dataset. *Inf. Softw. Technol.* **169**, 107413 (2024).
27. Mahmood, Y. et al. Software effort Estimation accuracy prediction of machine learning techniques: A systematic performance evaluation. *Software: Pract. Experience.* **52** (1), 39–65 (2022).
28. Sharma, S. & Vijayvargiya, S. An optimized neuro-fuzzy network for software project effort Estimation. *IETE J. Res.* **69** (10), 6855–6866 (2023).

Author contributions

Wang Long, Zhao Qixin and Yang Luxia wrote the main manuscript text. Wang Long, Zhao Qixin and Yang Luxia reviewed the manuscript.

Funding

(1) Shanxi Higher Education Laboratory Project, 2022P018, Independent innovation laboratory based on cross-five-dimensional multi-level intelligent big data. (2) Construction of the Innovation and Innovation Education and Training Certification Center. (3) Shanxi Province Key R&D Plan, Project Number: 202102010101008, Project Name: Public Transport Intelligent Monitoring System Based on 5G Communication.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to Y.L.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025